

A Machine-Checked Finite-Injury Priority Argument: The Friedberg–Muchnik Theorem in Lean 4

Ara Aslyan
Strike Technologies, New York
aaslyan@striketechologies.com

Abstract

The Friedberg–Muchnik theorem constructs two computably enumerable sets which are mutually nonreducible, giving the first solution to Post’s problem. Its classical proof is a finite-injury priority argument: it maintains evolving witnesses and restraints, initializes lower-priority strategies, and proves that every strategy is eventually stable. We describe a Lean 4 formalization of this argument in the repository `friedberg-muchnik-lean`. The development does not take oracle computation as a black-box semantic relation. Instead, it defines an explicit language of oracle programs and a step-indexed evaluator over finite oracle information. Successful bounded evaluations record both their output and a strict upper bound on the oracle positions queried; the central use theorem proves preservation of such computations under extra fuel and oracle agreement below the recorded use. On this foundation, the project formalizes the stage construction, the priority ordering, the attention predicate, initialization, restraints, and limit sets. Finite injury is proved by a rank argument after higher-priority requirements become quiet. A separate computability layer gives an encoded implementation of the stage function, proves it primitive recursive using Mathlib’s ordinary computability infrastructure, and derives that the two limits are computably enumerable. The final Lean theorem states

$$\exists AB : \text{Set } \mathbb{N}, CE(A) \wedge CE(B) \wedge \neg(A \leq_T B) \wedge \neg(B \leq_T A)$$

for the project’s local oracle-machine reducibility relation.

1 Introduction

Post’s problem asks whether there is a computably enumerable set whose Turing degree is neither computable nor complete. Friedberg and Muchnik answered this question independently by constructing two computably enumerable sets of incomparable degree [5, 6]. The method introduced in these proofs, now called the priority method, is one of the defining proof techniques of classical computability theory.

Priority arguments are difficult formalization targets because their textbook presentation relies on disciplined but informal temporal reasoning. A proof says to choose a fresh witness, wait until a computation converges, preserve the computation by restraining future enumeration below its use, initialize lower-priority strategies when a stronger strategy acts, and then argue that higher-priority strategies eventually stop acting. In a proof assistant these phrases must become explicit data and theorems: a state component for witnesses and restraints, a decidable attention predicate, a deterministic transition function, a preserved invariant, and an induction proving eventual stability.

This paper describes a Lean 4 formalization of the Friedberg–Muchnik theorem. The central contribution is not a new mathematical proof of the classical theorem, but a machine-checked account of the finite-injury method in which finite oracle computations, uses, restraints, initialization,

attention, injury, eventual stability, and effective stage construction are all represented directly. The development is organized around a project-local oracle-machine model, with Mathlib used only for ordinary primitive-recursive and partial-recursive closure obligations.

The final theorem is:

```
theorem friedberg_muchnik :
  exists A B : Set Nat,
    CE A /\ CE B /\ not (A <=T B) /\ not (B <=T A)
```

In the actual Lean source the relation is named `TuringReducible` and displayed with scoped notation $\leq^T$. The theorem is proved by taking the two limit sets `Aset` and `Bset`, proving them c.e. in `CE.lean`, and proving the two nonreducibilities in `Requirements.lean`.

The contributions are:

1. an explicit finite oracle semantics with output and use tracking;
2. a verified agreement-below-use preservation theorem;
3. a complete stage-by-stage formalization of the Friedberg–Muchnik priority construction;
4. a finite-injury proof based on a formal rank and induction over priority;
5. a computability proof showing that the limiting sets are genuinely c.e.; and
6. a final theorem stated in set-theoretic computability vocabulary.

The paper is careful about the scope of the local reducibility relation. The final theorem proves mutual nonreducibility in the project’s oracle-machine model. Reflexivity, the effective program numbering, and basic sanity lemmas are proved, but transitivity of the local reducibility relation is not currently part of the repository. We therefore treat the full internal development of Turing degrees as future work.

2 Mathematical Background

2.1 Computably Enumerable Sets

A computably enumerable set can be presented by an effective monotone sequence of finite approximations

$$A_0 \subseteq A_1 \subseteq A_2 \subseteq \dots$$

and the limit set $A = \bigcup_s A_s$. The monotonicity is positive: membership can change from absent to present when an element is enumerated, but never from present to absent.

This distinction matters for formalization. A prefix-monotone Boolean string representation would fix every bit once it appears in the prefix. Such a representation is appropriate for some finite-information arguments, but not for c.e. approximations themselves: a c.e. construction must be allowed to discover later that a previously absent number should be enumerated. In the Lean development, stage approximations are finite sets derived from lists, while Boolean lists are used only as bounded oracle snapshots for individual computations.

2.2 Oracle Computation and Use

For an oracle set B , the notation $\Phi_e^B(x)$ denotes the result of running the e -th oracle program on input x with access to B . A halting computation uses only finitely many oracle answers. Its *use* is an upper bound on the queried positions. The formalization uses the strict convention: if position p is queried, then the recorded use is at least $p + 1$, and query-free computations have use 0.

The use principle is the basis of restraint. If a computation halts with use u , then any later oracle agreeing below u produces the same finite computation and the same output. A priority strategy can therefore preserve a computation by preventing future enumeration below u .

2.3 The Friedberg–Muchnik Requirements

The construction builds two c.e. sets A and B . The requirements are interleaved by natural priority index:

$$R_e : A \neq \Phi_e^B, \quad S_e : B \neq \Phi_e^A,$$

with R_e represented by priority $2e$ and S_e by priority $2e + 1$. Lower numerical priority indices have higher priority.

The strategy for R_e chooses a fresh witness x , keeps $x \notin A$, and waits for a stage at which the current finite approximation to $\Phi_e^B(x)$ converges to 0. If that happens, it enumerates x into A , records the computation’s use as a restraint on B , and initializes all lower-priority requirements. The S_e strategy is the mirror image, exchanging A and B .

2.4 Finite Injury

The classical finite-injury proof proceeds by induction over priority. Once all higher-priority requirements stop acting, the current requirement can no longer be initialized from above. It then makes only finitely many progress steps: appoint a witness, possibly act, and then remain stable. The Lean proof makes this argument precise using the definitions **rank** and **reqRank**; after higher-priority requirements are quiet, each attention to the current requirement strictly increases a rank bounded by 2.

3 Design Choices

3.1 A Finitary Oracle Model

The project deliberately does not start from a high-level semantic type of oracle functionals such as $Set \mathbb{N} \rightarrow \mathbb{N} \rightarrow .\mathbb{N}$. Such a choice would require substantial infrastructure before the priority construction could begin: an oracle-machine model, continuity or compactness theorems, extraction of finite uses from semantic computations, and a connection between semantic functionals and executable indices.

Instead, finite execution is primary. The evaluator runs a concrete program for bounded fuel against a partial oracle $\mathbb{N} \rightarrow Option\ Bool$. Infinite computation is later defined by existence of a finite successful run against a total oracle. This representation puts exactly the finite information used by a priority construction into the main semantic object.

3.2 Finite Sets Versus Boolean Prefixes

The construction represents evolving c.e. approximations as monotone finite sets:

```

def StageMono (F : Nat -> Finset Nat) : Prop :=
  forall s, F s subset F (s + 1)

def limitSet (F : Nat -> Finset Nat) : Set Nat :=
  {n | exists s, n in F s}

```

The actual syntax uses Lean’s $\subseteq$ and $\in$ notation. Boolean lists appear separately as snapshots. This separation is one of the important design decisions: a stage set can enumerate new positive information at any later stage, while a snapshot merely supplies a finite window of answers to one bounded oracle computation.

3.3 Hybrid Use of Mathlib

The development separates project-local oracle reasoning from ordinary computability closure proofs. Project-local files define oracle syntax, bounded evaluation, infinite semantics, reducibility, the construction, invariants, finite injury, and satisfaction of requirements. Mathlib’s computability library enters through `MathlibBridge.lean`, `RunPrimrec.lean`, and the c.e. proof in `CE.lean`. This boundary allows the priority proof to reason about explicit finite uses while still reusing Mathlib’s closure theorems for primitive-recursive and partial-recursive functions [3, 1].

3.4 Local Reducibility Vocabulary

The central reducibility definition is:

```

def TuringReducible (A B : Set Nat) : Prop :=
  exists c : OracleCode,
    forall x : Nat,
      Computes c (charFun B) x (natOfBool (charFun A x))

```

Thus $A \leq_T B$ means that a project-local oracle program computes the characteristic function of A using the total characteristic function of B as oracle. The development proves reflexivity `TuringReducible.refl`, that computable sets reduce to every oracle via `ComputableSet.turingReducible`, and an effective bijection between natural numbers and `OracleCode`. It does not yet prove composition or transitivity of `TuringReducible`; that limitation is discussed in Section 10.

4 Formal Oracle Computation

4.1 Oracle Program Syntax

The file `Foundation/OracleCode.lean` defines the program language:

```

inductive OracleCode : Type where
  | zero
  | succ
  | left
  | right
  | query
  | pair (f g : OracleCode)
  | comp (f g : OracleCode)
  | prec (f g : OracleCode)
  | rfind (f : OracleCode)

```

The first four constructors are the usual initial functions on natural numbers. `pair`, `comp`, `prec`, and `rfind` give pairing, composition, primitive recursion, and unbounded search, in the same unary-with-pairing style used by Mathlib's `Nat.Partrec.Code`. The additional constructor `query` asks the oracle whether the input number is in the oracle set, returning 1 for true and 0 for false.

4.2 Partial Finite Oracles

The evaluator uses partial oracles:

```
abbrev PartOracle : Type := Nat -> Option Bool

def PartOracle.ofFun (X : Nat -> Bool) : PartOracle :=
  fun n => some (X n)

def PartOracle.ofSnapshot (sigma : List Bool) : PartOracle :=
  fun n => sigma[n]?
```

A total characteristic function is embedded by `ofFun`; a finite snapshot answers positions inside the list and returns `none` outside it. This makes insufficient oracle information a semantic case, not an accidental timeout.

4.3 Step-Indexed Evaluation and Results

Successful runs return both output and use:

```
structure HaltData where
  output : Nat
  use : Nat

inductive RunResult where
  | halt (d : HaltData)
  | stuck
  | timeout
```

The evaluator

```
def run : Nat -> PartOracle -> OracleCode -> Nat -> RunResult
```

is total in Lean because it is indexed by fuel. If the fuel runs out it returns `timeout`. If a query asks for a position where a finite snapshot has no answer, it returns `stuck`. If it halts, it returns `halt (output, use)`. The result type distinguishes three facts that the priority proof uses differently: convergence, insufficient finite oracle information, and insufficient fuel.

Use is propagated through sequencing by maximum. For a query at input x , the recorded use is $x + 1$. For query-free initial functions the use is 0. The file `FiniteEval.lean` provides named step lemmas such as `run_query_some`, `run_comp_step`, `run_prec_succ_step`, and `run_rfind_step`; later proofs rewrite through these equations rather than unfolding the evaluator directly.

4.4 The Use Theorem

The main preservation theorem is `run_halt_mono`:

```
theorem run_halt_mono
  (hk : k <= k')
  (h0 :
```

```

forall i, i < d.use ->
  forall b, 0 i = some b -> 0' i = some b)
(h : run k 0 c x = .halt d) :
run k' 0' c x = .halt d

```

The theorem says that a halting bounded run survives an increase of fuel and replacement of the oracle by one that preserves all answers actually used by the original computation. Notice that O' need not extend O everywhere. It only has to preserve answered positions below the recorded use; positions above the use and unanswered positions are irrelevant.

The proof follows the recursion defining `run`. In composition, primitive recursion, and minimization, the proof decomposes a halting `bind`, applies the induction hypothesis to subruns, and rebuilds the result while preserving the maximum-use bookkeeping. This is the formal version of the classical use principle: a computation already seen at a finite stage is preserved if future enumeration avoids its use.

4.5 Infinite Semantics

The file `Foundation/InfiniteEval.lean` defines:

```

def Computes (c : OracleCode) (X : Nat -> Bool)
  (x y : Nat) : Prop :=
exists k d,
  run k (PartOracle.ofFun X) c x = .halt d /\
  d.output = y

```

Thus infinite computation is recovered from existence of a finite successful run against the total oracle. The theorem `Computes.unique` gives determinism of outputs. The theorem `computes_iff_initialSegment` connects total-oracle computation to finite snapshots: a total computation exists iff some finite initial segment already suffices. This bridge is essential in the never-acted case of the requirement proof, where a hypothetical total computation must eventually become visible at a finite stage.

4.6 Program Numbering

`Foundation/Numbering.lean` defines an explicit coding `encodeCode : OracleCode -> Nat` and decoding `ofNatCode : Nat -> OracleCode`. The two round-trip theorems `ofNatCode_encodeCode` and `encodeCode_ofNatCode` prove that the numbering is a bijection. Requirements are indexed by natural numbers, and the final diagonalization converts an arbitrary candidate program c to its index `OracleCode.encodeCode c`; the round-trip theorem then shows that the corresponding requirement covers exactly c .

5 Finite Approximations and the Stage Construction

5.1 Approximations and Limits

The stage construction maintains lists of numbers already enumerated into the two sets. The finite-set approximations and limits are:

```

def AstageF (s : Nat) : Finset Nat :=
  (stageState s).Alist.toFinset

def BstageF (s : Nat) : Finset Nat :=

```

```

(stageState s).Blist.toFinset

def Aset : Set Nat := limitSet AstageF
def Bset : Set Nat := limitSet BstageF

```

The theorems `AstageF_mono` and `BstageF_mono` prove that these approximations are monotone. The generic approximation file proves `StageMono.stabilizesBelow`: below every finite bound, a monotone finite-stage approximation eventually agrees with its limit. The satisfaction proof uses this fact through `run_halt_snapshot_of_limit`.

5.2 Strategy State

Each requirement has a record:

```

structure ReqState where
  witness : Option Nat
  acted : Bool
  restraint : Nat

```

The construction state is:

```

structure ConsState where
  Alist : List Nat
  Blist : List Nat
  reqs : List ReqState
  fresh : Nat

```

The `fresh` counter is intended to be strictly above enumerated numbers and live witnesses and weakly above restraints. This intention is not left as a comment: it is part of the invariant `ConsInv` in `Invariants.lean`.

5.3 Attention and Action

Even priority $i = 2e$ attacks $A = \Phi_e^B$ and therefore queries the current B -list. Odd priority $i = 2e + 1$ attacks $B = \Phi_e^A$ and queries the current A -list. The definitions `oracleOf` and `targetOf` implement this parity split.

The convergence check is:

```

def convCheck (st : ConsState) (s i x : Nat) : Option Nat :=
  let v := nrun s (snapshotOf (st.oracleOf i) s) (i / 2) x
  if 2 <= v /\ (v - 2).unpair.1 = 0
  then some (v - 2).unpair.2 else none

```

It runs the numbered program $i/2$ with fuel s against a length- s snapshot of the current oracle side. If the encoded run halts with output 0, it returns the recorded use.

The attention predicate is:

```

def requiresAttention (st : ConsState) (s i : Nat) : Bool :=
  match st.req[i]? with
  | none => false
  | some r =>
    match r.witness with
    | none => true
    | some x => !r.acted && (st.convCheck s i x).isSome

```

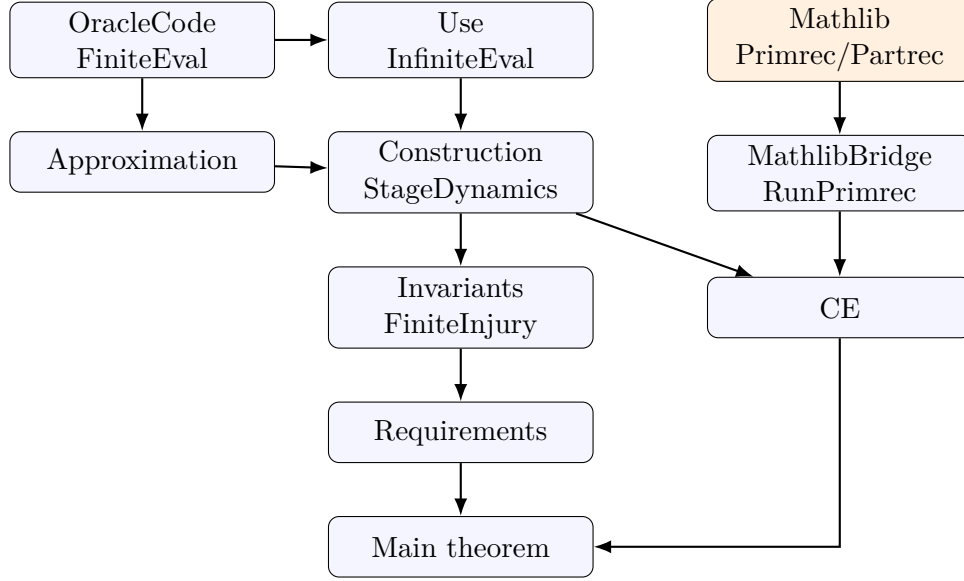


Figure 1: Repository architecture. Mathlib computability infrastructure enters through the bridge and c.e. computability layer; the oracle semantics and priority proof are project-local.

A requirement requires attention if its record exists and either no witness has been appointed, or it has an unacted witness with visible output 0. The deterministic stage step appends the newborn blank requirement record, finds the least requirement requiring attention, and calls `stepAt`. Appointment stores the fresh counter as witness; action enumerates the witness into the target side, records the use as restraint, and replaces every lower-priority record by `ReqState.init`.

5.4 Stage Dynamics and Invariants

`StageDynamics.lean` defines `preState` and `attended`, proves that `attended` is the least requirement requiring attention, and packages the transition in `step_cases`. It also proves `Alist_succ` and `Blist_succ`, which say that each side either stays unchanged or receives exactly the acting witness.

`Invariants.lean` introduces the ten-field invariant `ConsInv`. Its fields include freshness of enumerated numbers, freshness of witnesses, boundedness of restraints, coherence of blank records, distinctness of live witnesses, witness-respecting higher restraints, unacted witnesses outside both lists, acted witnesses in their target list, and preservation of the acted computation. The theorem `consInv_stage` proves that every reachable `stageState s` satisfies the invariant by induction over the three transition cases: record birth, appointment, and action.

6 Formal Finite Injury

The finite-injury proof is isolated in `FiniteInjury.lean`. Its goal is not merely to show that records eventually have stable values, but to provide the exact stabilization package consumed by the requirement satisfaction proofs.

6.1 Record Stability and Quietness

The theorem `record_step_stable` states that if requirement i 's record exists at stage s , and no requirement $j \leq i$ receives attention at that stage, then requirement i 's record is unchanged from

stage s to stage $s + 1$. The theorem `record_stable_of` extends this to an interval. This is the no-injury fact in its formal form: if nothing at or above a requirement acts, its record is frozen.

6.2 The Rank Argument

The local progress measure is:

```
def rank (r : ReqState) : Nat :=
  (if r.witness.isSome then 1 else 0) +
  (if r.acted then 1 else 0)

def reqRank (i s : Nat) : Nat :=
  rank ((stageState s).reqs[i]?).getD ReqState.init)
```

The theorem `rank_le_two` bounds every rank by 2. `reqRank_incr` proves that when requirement i receives attention, and its record already exists, its rank increases by exactly one. Appointment changes the witness bit from absent to present; action changes the acted bit from false to true. The theorem `reqRank_mono` proves that if all strictly higher-priority requirements are quiet at a step, the rank of i never decreases.

6.3 Induction Over Priority

The main theorem is:

```
theorem quiet_above (i : Nat) :
  exists s0,
    forall s, s0 <= s ->
      forall j, j <= i -> attended s != some j
```

The proof uses induction on i . Given a stage after which all requirements $j < i$ are quiet, suppose for contradiction that i receives attention arbitrarily late. Choose three later attention stages. Between them, higher priorities are quiet, so the rank cannot decrease; at each attention to i , the rank increases. This forces the rank above 2, contradicting `reqRank_le_two`. Hence i eventually stops receiving attention, and together with the induction hypothesis all requirements $j \leq i$ are eventually quiet.

The theorem `exists_stable` packages the conclusion: for every requirement i there are a stage s_1 and a record r such that $i < s_1$, the record at index i is always r after s_1 , and no requirement $j \leq i$ is ever attended after s_1 . Finally, `stable_not_requires` shows that after such a stage the requirement does not even require attention; otherwise the leastness theorem for `attended` would select some $j \leq i$.

6.4 Satisfaction of Requirements

`Requirements.lean` proves `R_satisfied` and `S_satisfied`. We describe the R_e case; S_e is symmetric. Let $i = 2e$, and take the stable record r and stage s_1 given by `exists_stable`. The proof first shows that r must contain a witness x ; otherwise the requirement would require attention by `requiresAttention_of_no_witness`, contradicting `stable_not_requires`.

If $r.acted = \text{true}$, then the invariant `acted_in` puts x in A , while `acted_comp` supplies a stored computation of program e against the current B -oracle snapshot with output 0 and use $r.restraint$. The theorem `restraint_respected_B` proves that no later enumeration below this restraint enters B , using the quiet tail and the invariant `wit_ge`. Then `run_halt_limit_of_restraint` transfers

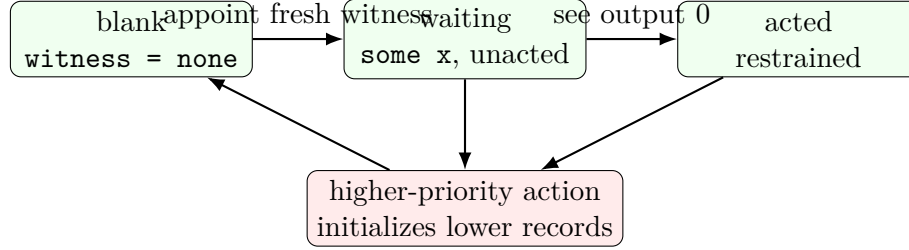


Figure 2: Lifecycle of one strategy. Higher-priority action initializes lower-priority records; once higher priorities are quiet, attention can only move the strategy forward through a bounded rank.

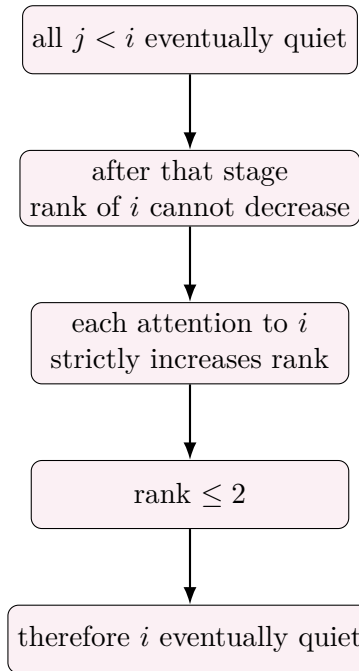


Figure 3: Finite-injury induction over priority.

the finite computation to the limiting oracle $Bset$. Thus the program outputs 0 at x , while the characteristic function of $Aset$ outputs 1.

If $r.acted = false$, the invariant `unacted_out` shows that x never enters $Aset$. If a candidate reduction computed $Aset$ from $Bset$, it would output 0 at x . By `run_halt_snapshot_of_limit`, this convergent computation would be visible against all sufficiently late finite snapshots. Choosing a stage large enough for the fuel and use, the helper `run_halt_toFinset_snapshot` converts that visibility into `convCheck`, so the requirement would again require attention, contradicting `stable_not_requires`.

The final nonreducibility theorems are:

```
theorem not_A_le_B : not (TuringReducible Aset Bset)
theorem not_B_le_A : not (TuringReducible Bset Aset)
```

They use the program numbering: an arbitrary candidate program c is assigned index `OracleCode.encodeCode c`, and the round-trip theorem `OracleCode.ofNatCode_encodeCode` identifies the corresponding requirement with c .

7 Proving Computable Enumerability

The final theorem requires more than abstract monotone limits. The sets `Aset` and `Bset` must satisfy the repository definition `CE`, i.e. each must be the halting domain of an oracle-free local program. The file `CE.lean` proves this by combining a computability-friendly encoded construction with the Mathlib bridge.

7.1 Encoded Construction

The mathematical construction state contains custom structures `ReqState` and `ConsState`. Rather than building custom `Primcodable` instances for these types, `CE.lean` defines a parallel tuple representation:

```
abbrev RS : Type := Option Nat * Bool * Nat
abbrev St : Type := List Nat * List Nat * List RS * Nat

def encReq (r : ReqState) : RS :=
  (r.witness, r.acted, r.restraint)

def encSt (st : ConsState) : St :=
  (st.Alist, st.Blist, st.reqs.map encReq, st.fresh)
```

It then defines tuple-level versions of the convergence check, attention predicate, idle step, appointment step, action step, stage step, and full stage recursion: `convCheckN`, `reqAttN`, `idleSt`, `appointSt`, `actSt`, `stepN`, and `stageN`.

7.2 Simulation

The key simulation theorem is:

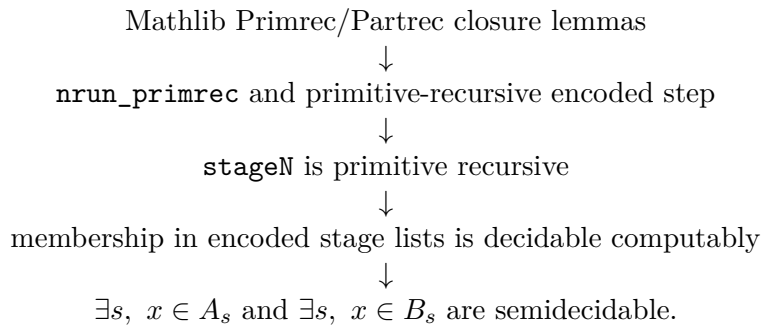
```
theorem encSt_stepState (st : ConsState) (s : Nat) :
  encSt (stepState st s) = stepN (encSt st) s
```

By induction, `encSt_stage` proves that the encoded construction computes the encoding of the mathematical construction at every stage. The lemmas `mem_Alist_iff_stageN` and `mem_Blist_iff_stageN` transfer membership facts between the two representations.

Two list-surgery lemmas, `map_range_set` and `map_range_act`, explain how list update and lower-priority initialization are represented as maps over `List.range`. This representation makes the operations amenable to Mathlib’s primitive-recursive list combinators.

7.3 Primitive Recursiveness and Semidecision

`CE.lean` proves primitive recursiveness of the encoded transition and stage function. The proof depends on `nrun_primrec`, the primitive-recursive evaluator on numbered oracle codes and finite snapshots. The chain of results is:



The final c.e. theorems are:

```
theorem ce_Aset : CE Aset
theorem ce_Bset : CE Bset
```

Each proof builds a Mathlib partial-recursive search over stages using `Partrec.rfind`, then applies `ce_of_partrec_dom` from `MathlibBridge.lean` to obtain the repository-local CE statement.

8 Artifact Evaluation and Proof Engineering

8.1 Verified Artifact Facts

The following facts were checked locally in the repository before drafting. The command `lake build` completed successfully:

Build completed successfully (811 jobs).

The final theorem and axiom footprint were checked with:

```
import FriedbergMuchnik.Main

#check FriedbergMuchnik.friedberg_muchnik
#print axioms FriedbergMuchnik.friedberg_muchnik
```

Lean reported that `friedberg_muchnik` depends on `propept`, `Classical.choice`, and `Quot.sound`. No additional project axiom appears in the final theorem’s footprint.

Item	Value
Lean toolchain	leanprover/lean4:v4.26.0
Mathlib revision	2df2f0150c275ad53cb3c90f7c98ec15a56a1a67
Lean files under FriedbergMuchnik plus root	17
Lean source lines, including comments/blank lines	4024
Top-level declarations counted by regex	178
lake build	succeeds locally
Final theorem axiom footprint	propext, Classical.choice, Quot.sound
sorry/admit/escape-hatch grep	no source declarations found
Repository license	no LICENSE file found
CI workflow	no .github workflow found

Table 1: Artifact facts verified from the repository.

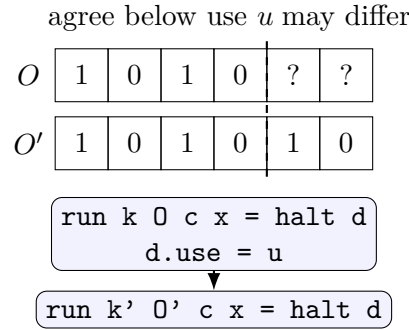


Figure 4: Use preservation: oracles may differ above the recorded use, but agreement below it preserves the same bounded computation.

8.2 Proof-Engineering Lessons

The development illustrates several representation lessons.

First, c.e. approximations should be represented as finite positive information, not as prefix-fixed Boolean strings. The latter representation would obscure the basic monotonicity of enumeration and risks proving the wrong theorem.

Second, use should be recorded by the evaluator itself. Proving after the fact that every successful computation has some finite use would add a continuity argument at exactly the point where the priority construction needs executable finite data.

Third, finite and infinite semantics should be separated. The construction acts only on finite snapshots and bounded runs; the final theorem speaks about total characteristic-function oracles. The bridge lemmas in `InfiniteEval.lean` and `Approximation.lean` connect these layers without mixing them.

Fourth, finite injury becomes tractable when the informal phrase "injured finitely often" is replaced by a bounded rank. The rank 0, 1, 2 is small enough for arithmetic proof but expressive enough to capture appointment and action.

Finally, the c.e. proof benefits from a second implementation of the stage function. The mathematical state is convenient for the priority proof; the tuple state is convenient for primitive-recursive closure. The simulation theorem keeps these two views synchronized.

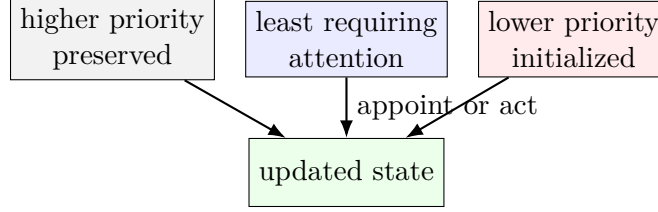


Figure 5: One priority stage preserves higher-priority records, acts for the least requirement needing attention, and initializes lower-priority records.

9 Related Work

Friedberg’s 1957 paper constructed two recursively enumerable sets of incomparable degrees and solved Post’s problem [5]. Muchnik independently obtained the same solution in 1956, with an expanded publication appearing later [6, 7]. Soare’s monograph remains a standard reference for recursively enumerable sets, degrees, and priority methods [9]. The formalization follows the finite-injury presentation rather than a reformulation that avoids priority machinery.

The Lean development builds on Mathlib’s ordinary computability theory. Carneiro formalized primitive recursive and partial recursive functions in Lean, including universal partial recursive functions and the halting problem [3]. Mathlib’s current `Partrec` and `PartrecCode` libraries provide the closure theorems and code infrastructure reused here [1, 2]. The present project differs by adding a local relative oracle-program semantics with explicit finite uses, which is tailored to priority reasoning.

Other proof assistants also contain computability developments. The Isabelle Archive of Formal Proofs includes a universal Turing machine formalization with undecidability and computability-theoretic material [10], building on Isabelle/HOL’s mature theorem-proving infrastructure [8]. Coq has been used to formalize abstract computability through Turing categories [4]. These works address foundational computability models; the contribution here is a direct mechanization of a finite-injury priority argument.

We are not aware of another publicly available Lean formalization of the complete Friedberg–Muchnik finite-injury construction. This claim should be treated cautiously: a submission should include a more systematic search of formalization archives and unpublished developments before making a strong priority claim.

10 Limitations and Future Work

10.1 Transitivity of Local Reducibility

The repository proves the mutual nonreducibility theorem it states, but it does not yet develop the full algebra of the local reducibility relation. In particular, transitivity of `TuringReducible` is not currently proved. The final theorem does not require transitivity: it directly shows that no local oracle program computes $Aset$ from $Bset$, and no local oracle program computes $Bset$ from $Aset$.

For an internal theory of Turing degrees, transitivity should be added. A natural route is query substitution: given a program computing A from B and a program computing B from C , compile a new oracle program which replaces each query to B by the second program run against C . One must then prove correctness and use preservation for this compilation.

10.2 Equivalence With Other Semantics

The project defines its own oracle-machine model. It proves an embedding of ordinary Mathlib partial recursive codes into the local query-free fragment, but it does not prove equivalence with a separate established relative-computability framework. Such a theorem would strengthen the connection between the local model and other formalizations of Turing reducibility.

10.3 Reusable Priority Framework

The finite-injury proof is specialized to Friedberg–Muchnik. Future work could abstract the common ingredients: requirements, strategies, priority order, initialization, restraints, and finite progress measures. This could support formalizations of simple sets, splitting theorems, permitting arguments, minimal pairs, or infinite-injury constructions. The current development should be seen as a case study and foundation, not a general priority library.

10.4 Artifact Packaging

No repository license or CI workflow was found at drafting time. A submission artifact should add a license, reproducible build instructions, and continuous integration running at least `lake build` and the axiom-footprint check.

11 Conclusion

The formalization demonstrates that a classical finite-injury priority argument can be represented directly in Lean 4. The proof turns informal temporal reasoning into finite executable stages, explicit state components, preserved invariants, recorded semantic uses, bounded progress, and machine-checked stabilization.

The central methodological lesson is that priority arguments are not inherently beyond current interactive theorem provers, but they require representations that expose finite information. In this development, finite oracle snapshots coexist with monotone positive stage sets; successful computations carry their own uses; finite stage semantics and infinite oracle semantics are bridged by explicit theorems; and finite injury is reduced to a small rank argument after higher priorities become quiet.

The final theorem establishes two computably enumerable limit sets which are mutually nonreducible in the project’s oracle-machine model. Extending the local reducibility relation with transitivity and connecting it to other relative-computability semantics would make the degree-theoretic interpretation fully internal. Even in its present form, the artifact provides a detailed machine-checked account of the finite-injury method and a basis for further formalized computability theory.

References

- [1] Mathlib documentation: Mathlib.computability.partrec. https://leanprover-community.github.io/mathlib4_docs/Mathlib/Computability/Partrec.html, 2026. Accessed 2026-07-26.
- [2] Mathlib documentation: Mathlib.computability.partreccode. https://leanprover-community.github.io/mathlib4_docs/Mathlib/Computability/PartrecCode.html, 2026. Accessed 2026-07-26.

- [3] Mario Carneiro. Formalizing computability theory via partial recursive functions. In *10th International Conference on Interactive Theorem Proving (ITP 2019)*, volume 141 of *Leibniz International Proceedings in Informatics*, pages 12:1–12:17. Schloss Dagstuhl–Leibniz-Zentrum für Informatik, 2019.
- [4] J. R. B. Cockett, Pieter Hofstra, and Pavel Hrubes. Formalizing abstract computability: Turing categories in coq. *Electronic Notes in Theoretical Computer Science*, 338:203–218, 2018.
- [5] Richard M. Friedberg. Two recursively enumerable sets of incomparable degrees of unsolvability (solution of post’s problem, 1944). *Proceedings of the National Academy of Sciences of the United States of America*, 43(2):236–238, 1957.
- [6] A. A. Muchnik. On the unsolvability of the problem of reducibility in the theory of algorithms. *Doklady Akademii Nauk SSSR*, 108:194–197, 1956. In Russian.
- [7] A. A. Muchnik. Solution of post’s reduction problem and of certain other problems in the theory of algorithms. *Trudy Moskovskogo Matematicheskogo Obshchestva*, 7:391–405, 1958.
- [8] Tobias Nipkow, Lawrence C. Paulson, and Markus Wenzel. *Isabelle/HOL: A Proof Assistant for Higher-Order Logic*, volume 2283 of *Lecture Notes in Computer Science*. Springer, 2002.
- [9] Robert I. Soare. *Recursively Enumerable Sets and Degrees: A Study of Computable Functions and Computably Generated Sets*. Perspectives in Mathematical Logic. Springer, 1987.
- [10] Jian Xu, Xingyuan Zhang, Christian Urban, Sebastiaan J. C. Joosten, and Franz Regensburger. Universal turing machine. Archive of Formal Proofs, 2019. https://www.isa-afp.org/entries/Universal_Turing_Machine.html.

A Textbook-to-Lean Map

B Artifact Reproduction

The repository is available at:

<https://github.com/aaslyan/friedberg-muchnik-lean>

A fresh build should use:

```
git clone https://github.com/aaslyan/friedberg-muchnik-lean
cd friedberg-muchnik-lean
lake exe cache get
lake build
```

To check the final theorem and axiom footprint:

```
cat >/tmp/check_fm.lean <<'EOF'
import FriedbergMuchnik.Main

#check FriedbergMuchnik.friedberg_muchnik
#print axioms FriedbergMuchnik.friedberg_muchnik
EOF

lake env lean /tmp/check_fm.lean
```


Textbook concept	Lean representation	Main declaration	File
Oracle program	program syntax	OracleCode	Foundation/OracleCode.lean
Finite computation	bounded evaluator	run	Foundation/FiniteEval.lean
Output/use	halting data	HaltData	Foundation/FiniteEval.lean
Use preservation	agreement below use	run_halt_mono	Foundation/Use.lean
Infinite computation	existential finite run	Computes	Foundation/InfiniteEval.lean
Program enumeration	code numbering	ofNatCode_encodeCode	Foundation/Numbering.lean
c.e. approximation	monotone stages	StageMono	Approximation.lean
Limit set	union of stages	limitSet	Approximation.lean
Requirement state	strategy record	ReqState	Construction.lean
Witness	strategy field	witness	Construction.lean
Restraint	strategy field	restraint	Construction.lean
Attention	stage predicate	requiresAttention	Construction.lean
Priority action	deterministic step	stepAt, stepState	Construction.lean
No upward injury	record preservation	reqs_getElem?_succ_of_lt	StageDynamics.lean
Invariant package	construction invariant	ConsInv, consInv_stage	Invariants.lean
Finite injury	eventual quietness	quiet_above	FiniteInjury.lean
Stabilization	final record	exists_stable	FiniteInjury.lean
Diagonalization	requirement satisfaction	R_satisfied, S_satisfied	Requirements.lean
c.e. limit set	semidecision	ce_Aset, ce_Bset	CE.lean
Final theorem	assembly	friedberg_muchnik	Main.lean

Table 2: Mapping between textbook proof concepts and Lean declarations.

At drafting time this reported:

```
FriedbergMuchnik.friedberg_muchnik :
  exists A B,
    FriedbergMuchnik.CE A /\
      FriedbergMuchnik.CE B /\
        not FriedbergMuchnik.TuringReducible A B /\
          not FriedbergMuchnik.TuringReducible B A
'FriedbergMuchnik.friedberg_muchnik' depends on axioms:
[proptext, Classical.choice, Quot.sound]
```

To search for common proof escape hatches:

```
rg -n "sorry|admit|axiom|unsafe|opaque|extern|implemented_by|native_decide" \
  FriedbergMuchnik FriedbergMuchnik.lean lakefile.lean README.md STATUS.md
```

At drafting time this command found only documentation mentions of `sorry/axiom`, not source declarations.